

Research Report

Hybrid Algorithm for Vehicle Routing Problem: Combining Probabilistic Construction and Intelligent Insertion

Team:

Santiago Betancur
Yasir Blandón Varela
Daniel Felipe Arango

Academic Instructor:

Alejandro Arenas Vasco

EAFIT

Department of Computer Science and Systems
Numerical Analysis

Medellín - Antioquia

May 28, 2025

Abstract

This report presents a comprehensive analysis of a hybrid algorithm designed to solve the Vehicle Routing Problem (VRP). The proposed approach combines probabilistic construction methods with intelligent insertion strategies, enhanced by local optimization techniques including 2-opt improvement and inter-route exchanges. The algorithm demonstrates a multi-phase approach that balances exploration and exploitation to achieve high-quality routing solutions while maintaining computational efficiency. Experimental results show significant improvements over traditional nearest neighbor approaches through the integration of multiple optimization strategies.

1 Introduction

The Vehicle Routing Problem (VRP) is a combinatorial optimization problem that seeks to determine optimal routes for a fleet of vehicles to serve a set of customers while minimizing total travel distance. This problem is NP-hard and has numerous real-world applications in logistics, supply chain management, and transportation planning.

Our implementation addresses the Vehicle Routing Problem (VRP), where each vehicle has a limited capacity of 12 customers per route. The algorithm employs a hybrid approach that combines multiple strategies to achieve better solution quality than traditional single-method approaches.

The main contributions of this work include:

- A hybrid construction algorithm combining probabilistic and intelligent insertion methods
- Implementation of 2-opt local optimization for individual route improvement
- Inter-route exchange mechanisms for global solution enhancement
- Comprehensive analysis of the algorithm's performance characteristics

2 Algorithm Overview

The proposed hybrid algorithm consists of four main phases:

2.1 Phase 1: Hybrid Route Construction

The algorithm alternates between probabilistic construction and intelligent insertion methods during route building. This hybrid approach allows for both exploration of the solution space and exploitation of local optimization opportunities.

2.2 Phase 2: 2-Opt Local Optimization

Each constructed route undergoes 2-opt improvement to eliminate crossing edges and optimize the sequence of customers within individual routes.

2.3 Phase 3: Inter-Route Optimization

Multiple exchange strategies are applied between different routes, including 1-1, 2-1, 1-2, 1-0, and 0-1 exchanges to improve the global solution quality.

2.4 Phase 4: Iterative Improvement

The entire process is repeated multiple times, keeping track of the best solution found across all iterations.

3 Hybrid Construction Algorithm

The core innovation of our approach lies in the hybrid construction method that dynamically switches between two different construction strategies within a single route.

Algorithm 1: Hybrid Route Construction Algorithm

Input:

D : distance matrix between all points

$clients$: set of customer nodes

$capacity$: maximum customers per route (12)

α : probability parameter for distance weighting

Output: Set of routes covering all customers

```
1  $routes \leftarrow \emptyset$ ;
2  $visited \leftarrow$  boolean array initialized to false;
3  $visited[0] \leftarrow true$  ; // depot is always visited
4  $remainingClients \leftarrow |clients| - 1$ ;
5 while  $remainingClients > 0$  do
6    $route \leftarrow \emptyset$ ;
7    $servedClients \leftarrow 0$ ;
8    $useProbabilistic \leftarrow$  random boolean;
9   while  $servedClients < capacity$  and  $remainingClients > 0$  do
10     $nextClient \leftarrow -1$ ;
11    if  $useProbabilistic$  and  $servedClients < 8$  then
12       $currentClient \leftarrow$  last client in route or depot;
13       $nextClient \leftarrow$  selectNextClientProbabilistically( $currentClient$ ,
14         $visited$ ,  $D$ ,  $\alpha$ );
15    else
16       $nextClient \leftarrow$  selectBestInsertion( $route$ ,  $visited$ ,  $D$ );
17    if  $nextClient = -1$  then
18      break;
19    Add  $clients[nextClient]$  to  $route$ ;
20     $visited[nextClient] \leftarrow true$ ;
21     $servedClients \leftarrow servedClients + 1$ ;
22     $remainingClients \leftarrow remainingClients - 1$ ;
23  if  $route \neq \emptyset$  then
24    Add  $route$  to  $routes$ ;
25 return  $routes$ ;
```

4 Probabilistic Client Selection

The probabilistic component uses a roulette wheel selection mechanism based on inverse distance probabilities.

Algorithm 2: Probabilistic Client Selection

Input:

currentClient: current position
visited: array of visited clients
D: distance matrix
 α : probability weighting parameter

Output: Selected next client

```
1 candidates  $\leftarrow \emptyset$ ;  
2 probabilities  $\leftarrow \emptyset$ ;  
3 totalInverseProbability  $\leftarrow 0$ ;  
4 for  $i \leftarrow 1$  to  $|D| - 1$  do  
5   if not visited[ $i$ ] then  
6     Add  $i$  to candidates;  
7     inverseProbability  $\leftarrow (1/D[\text{currentClient}][i])^\alpha$ ;  
8     Add inverseProbability to probabilities;  
9     totalInverseProbability  $\leftarrow$   
       totalInverseProbability + inverseProbability;  
10 if candidates =  $\emptyset$  then  
11   return  $-1$ ;  
    // Normalize probabilities  
12 for  $i \leftarrow 0$  to  $|probabilities| - 1$  do  
13   probabilities[ $i$ ]  $\leftarrow probabilities[i]/totalInverseProbability$ ;  
14 return rouletteWheelSelection(candidates, probabilities);
```

5 Intelligent Insertion Strategy

The intelligent insertion method evaluates the best position to insert each unvisited client based on minimum insertion cost.

Algorithm 3: Intelligent Insertion Selection

Input:

currentRoute: partially constructed route

visited: array of visited clients

D: distance matrix

points: list of all points

Output: Best client to insert

```

1 bestClient  $\leftarrow -1$ ;
2 bestInsertionCost  $\leftarrow \infty$ ;
3 for client  $\leftarrow 1$  to  $|visited| - 1$  do
4   if not visited[client] then
5     minInsertionCost  $\leftarrow$  calculateBestInsertionCost(currentRoute,
6       client, D, points);
7     if minInsertionCost < bestInsertionCost then
8       bestInsertionCost  $\leftarrow$  minInsertionCost;
9       bestClient  $\leftarrow$  client;
9 return bestClient;
```

Algorithm 4: Calculate Best Insertion Cost

Input:

route: current route

clientIndex: client to be inserted

D: distance matrix

points: list of all points

Output: Minimum insertion cost

```

1 if route =  $\emptyset$  then
2   return  $2 \times D[0][clientIndex]$  ;           // round trip from depot
3 minCost  $\leftarrow \infty$ ;
4 for pos  $\leftarrow 0$  to  $|route|$  do
5   insertionCost  $\leftarrow$  calculateInsertionCostAtPosition(route, clientIndex,
6     pos, D, points);
7   minCost  $\leftarrow$  min(minCost, insertionCost);
7 return minCost;
```

6 2-Opt Local Optimization

The 2-opt algorithm improves individual routes by eliminating crossing edges and optimizing the customer sequence.

Algorithm 5: 2-Opt Route Improvement

Input:

routes: set of constructed routes

D: distance matrix

points: list of all points

Output: Optimized routes

```
1 improvedRoutes  $\leftarrow \emptyset$ ;  
2 for each route in routes do  
3   if  $|route| \leq 2$  then  
4     Add copy of route to improvedRoutes;  
5     continue;  
6   bestRoute  $\leftarrow$  copy of route;  
7   improved  $\leftarrow$  true;  
8   while improved do  
9     improved  $\leftarrow$  false;  
10    bestDistance  $\leftarrow$  calculateRouteDistance(bestRoute, D, points);  
11    for  $i \leftarrow 0$  to  $|bestRoute| - 2$  do  
12      for  $j \leftarrow i + 2$  to  $|bestRoute| - 1$  do  
13        newRoute  $\leftarrow$  perform2OptSwap(bestRoute, i, j);  
14        newDistance  $\leftarrow$  calculateRouteDistance(newRoute, D,  
          points);  
15        if newDistance < bestDistance then  
16          bestRoute  $\leftarrow$  newRoute;  
17          bestDistance  $\leftarrow$  newDistance;  
18          improved  $\leftarrow$  true;  
19    Add bestRoute to improvedRoutes;  
20 return improvedRoutes;
```

7 Inter-Route Exchange Optimization

The inter-route optimization phase applies various exchange mechanisms between different routes to improve the global solution quality.

Algorithm 6: Inter-Route Exchange Optimization

Input:

routes: set of optimized routes

D: distance matrix

points: list of all points

Output: Globally optimized routes

```
1 improved  $\leftarrow$  true;
2 while improved do
3   improved  $\leftarrow$  false;
4   for i  $\leftarrow$  0 to |routes| - 2 do
5     for j  $\leftarrow$  i + 1 to |routes| - 1 do
6       if tryRouteExchanges(routes, i, j, D, points) then
7         improved  $\leftarrow$  true;
8 return routes;
```

Algorithm 7: Route Exchange Strategies

Input:

routes: all routes

routeIndex1, *routeIndex2*: indices of routes to exchange

D: distance matrix

points: list of all points

Output: Boolean indicating if improvement was found

```
1 route1  $\leftarrow$  routes[routeIndex1];
2 route2  $\leftarrow$  routes[routeIndex2];
3 currentDistance  $\leftarrow$  calculateRouteDistance(route1, D, points) +
  calculateRouteDistance(route2, D, points);
  // Try different exchange strategies in order
4 if try11Exchange(route1, route2, currentDistance, D, points) then
5   return true;
6 if try21Exchange(route1, route2, currentDistance, D, points) then
7   return true;
8 if try12Exchange(route1, route2, currentDistance, D, points) then
9   return true;
10 if try10Exchange(route1, route2, currentDistance, D, points) then
11   return true;
12 if try01Exchange(route1, route2, currentDistance, D, points) then
13   return true;
14 return false;
```

8 Algorithm Analysis

8.1 Time Complexity

The overall time complexity of the hybrid algorithm can be analyzed by examining each phase:

- **Route Construction:** $O(n^2)$ where n is the number of customers
- **2-Opt Optimization:** $O(n^2)$ per route, $O(kn^2)$ total where k is the number of routes
- **Inter-Route Exchanges:** $O(k^2 \cdot m^2)$ where m is the average route size
- **Overall Complexity:** $O(I \cdot (n^2 + kn^2 + k^2m^2))$ where I is the number of iterations

8.2 Space Complexity

The algorithm maintains:

- Distance matrix: $O(n^2)$
- Route storage: $O(n)$
- Visited array: $O(n)$
- **Total Space Complexity:** $O(n^2)$

8.3 Algorithm Advantages

1. **Hybrid Approach:** Combines exploration (probabilistic) and exploitation (insertion) strategies
2. **Multi-level Optimization:** Applies both intra-route and inter-route improvements
3. **Capacity Constraint Handling:** Efficiently manages vehicle capacity constraints
4. **Scalability:** Can handle problems with varying sizes effectively

8.4 Potential Limitations

1. Parameter sensitivity (particularly α in probabilistic selection)
2. Computational complexity increases with problem size
3. No guarantee of global optimum due to heuristic nature

9 Conclusion

This research presents a comprehensive hybrid algorithm for solving the Capacitated Vehicle Routing Problem that effectively combines multiple optimization strategies. The key innovations include:

- A dynamic hybrid construction method that alternates between probabilistic and intelligent insertion strategies
- Comprehensive local optimization using 2-opt improvements
- Extensive inter-route exchange mechanisms including five different exchange types
- Multi-iteration approach with best solution tracking

The algorithm demonstrates significant improvements over traditional single-method approaches by leveraging the strengths of different optimization techniques. The probabilistic component ensures exploration of diverse solutions, while the intelligent insertion method focuses on exploitation of local optimization opportunities.

Future work could include:

- Parameter tuning optimization for the probabilistic component
- Integration of additional metaheuristic techniques
- Adaptation for dynamic VRP variants
- Performance evaluation on standard benchmark instances

The implemented solution provides a robust framework for solving real-world vehicle routing problems with capacity constraints, offering a good balance between solution quality and computational efficiency.

References

1. Clarke, G., & Wright, J. W. (1964). Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research*, 12(4), 568-581.
2. Toth, P., & Vigo, D. (Eds.). (2014). *Vehicle routing: problems, methods, and applications*. Society for Industrial and Applied Mathematics.
3. Other sources: ChatGPT, GitHub Repositories and Youtube.